Experiment 30: Prolog Program for backward Chaining. Incorporate required queries.

Aim: To write a Prolog program that demonstrates backward chaining using facts, rules, and queries.

Algorithm:

1. Start the SWI-Prolog environment.
2. Define the facts rainy and cold.
3. Define rules to derive wet_ground, slippery, and wear_jacket.
4. Load the Prolog program using the consult/1 command.
5. Enter a query to check the required conclusion.
6. Prolog searches for a matching rule and checks the conditions recursively.
7. If all conditions are satisfied, Prolog returns true; otherwise, it returns false.
8. Stop the execution.

Code:

backward_chaining.pl

```
% Facts
fact(rainy).
fact(cold).

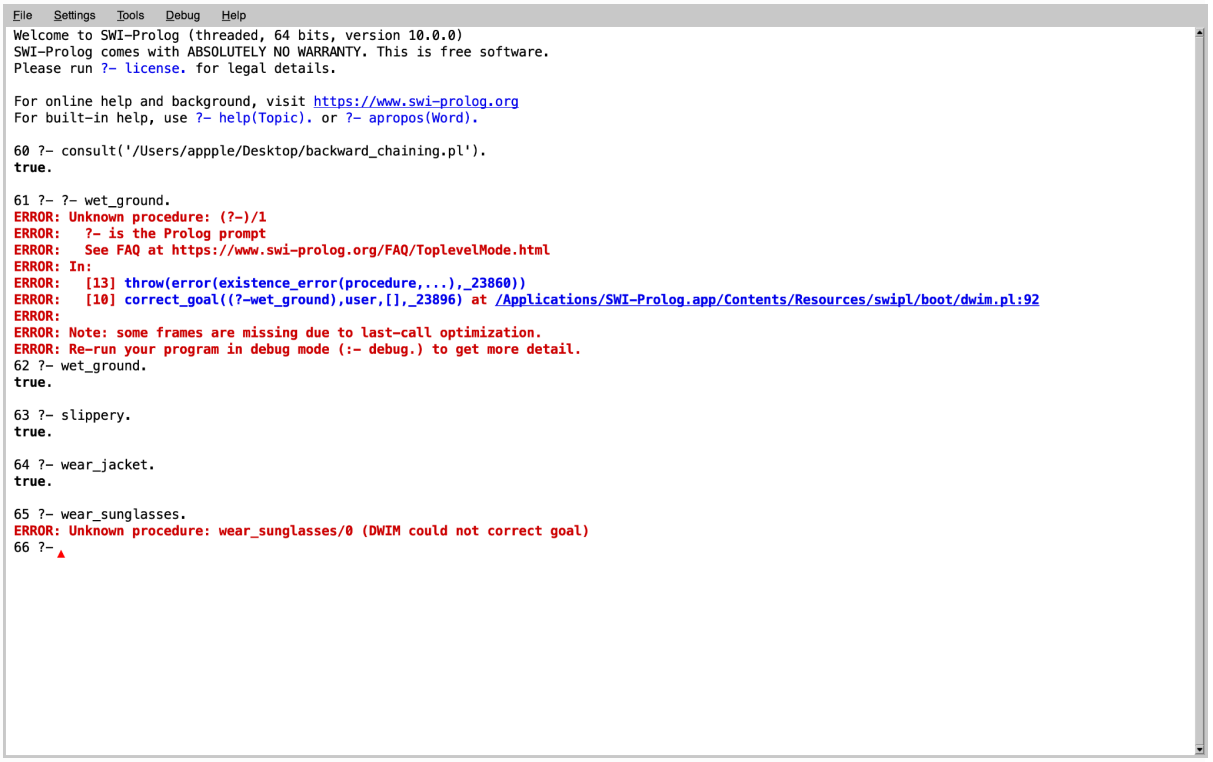
% Rules
% If it is rainy, the ground is wet
wet_ground :- fact(rainy).

% If the ground is wet, it is slippery
slippery :- wet_ground.

% If it is cold and rainy, wear a jacket
wear_jacket :- fact(cold), fact(rainy).

% Backward chaining queries are given below.
```

Output:



Result: Thus, the Prolog program for backward chaining was successfully implemented and executed. The required queries were tested, and Prolog determined whether the conclusions could be proved using the given facts and rules.

Experiment 31: Prolog program to implement pattern matching

Aim: To write and execute a Prolog program to implement pattern matching using facts and rules.

Algorithm:

1. Start the SWI-Prolog environment.
2. Define facts representing students and their departments.
3. Define the rule same_department/2 to check whether two students belong to the same department.
4. Load the program using the consult/1 command.
5. Enter queries to test pattern matching.
6. Prolog matches the query with available facts and rules.
7. Display true if a match is found, or false otherwise.
8. Stop execution.

Code:

pattern_matching.pl

backward_chaining.pl | pattern_matching.pl

```
% Pattern Matching in Prolog

% Facts
student(sneha, ece).
student(rahul, cse).
student(priya, eee).

% Rules
same_department(X, Y) :-
    student(X, D),
    student(Y, D),
    X \= Y.

% Queries
% ?- student(sneha, ece).
% ?- student(rahul, cse).
% ?- same_department(sneha, rahul).
% ?- same_department(sneha, priya).
% ?- student(sneha, Department).
```

Output:

```
66 ?- consult('/Users/appple/Desktop/pattern_matching.pl').
true.

67 ?- ?- student(sneha, ece).
ERROR: Unknown procedure: (?-)/1
ERROR:   ?- is the Prolog prompt
ERROR:   See FAQ at https://www.swi-prolog.org/FAQ/ToplevelMode.html
ERROR: In:
ERROR:   [13] throw(error(existence_error(procedure,...),_8394))
ERROR:   [10] correct_goal((?-student(sneha,ece)),user,[],_8430) at /Applications/SWI-Prolog.app/Contents/Resources/swipl/boot/dwim.pl:92
ERROR:
ERROR: Note: some frames are missing due to last-call optimization.
ERROR: Re-run your program in debug mode (:- debug.) to get more detail.
68 ?- student(sneha, ece).
true.

69 ?- same_department(sneha, rahul).
false.

70 ?- student(sneha, Department).
Department = ece.
```

Result: Thus, the Prolog program for pattern matching was implemented and executed successfully. The queries demonstrated matching of facts, variable binding, and rule evaluation.

Experiment 32: Prolog program to find the number of vowels

Aim: To write and execute a Prolog program to count the number of vowels in a given word.

Algorithm:

1. Start the SWI-Prolog environment.
2. Define the vowels a, e, i, o, and u as facts.
3. Define the base case: an empty list contains zero vowels.
4. Check the first character of the list.
5. If it is a vowel, recursively count the remaining vowels and add one.
6. If it is not a vowel, recursively count the remaining vowels without increasing the count.
7. Display the total number of vowels.
8. Stop execution.

Code:

count_vowels.pl

backward_chaining.pl | pattern_matching.pl | count_vowels.pl

```
% Prolog program to count vowels

vowel(a).
vowel(e).
vowel(i).
vowel(o).
vowel(u).

count_vowels([], 0).

count_vowels([H|T], Count) :-
    vowel(H),
    count_vowels(T, Rest),
    Count is Rest + 1.

count_vowels([H|T], Count) :-
    \+ vowel(H),
    count_vowels(T, Count).
```

Output:

```
71 ?- consult('/Users/appple/Desktop/count_vowels.pl').
true.

72 ?- count_vowels([h,e,l,l,o], Count).
Count = 2 .

73 ?- count_vowels([a,p,p,l,e], Count).
Count = 2 .

74 ?- count_vowels([p,r,o,l,o,g], Count).
Count = 2 .

75 ?- count_vowels([], Count).
Count = 0.
```

Result:

Thus, the Prolog program to count vowels in a given list of characters was implemented and executed successfully. It returns the total number of vowels present in the input list.